Employee Management Page

Frontend Engineer · Technical Assessment

React + TypeScript | No backend required | Design is not the focus

Design is not a focus of this task. Use any UI library, component library, CSS framework, or styling approach you prefer.

THE PROBLEM STATEMENT

HR teams need a simple way to quickly find and manage employees. Your goal is to build an Employee Management Page that allows HR users to search, filter, view, add, edit, and deactivate employees using React and TypeScript.

What you must deliver

Four client submissions from the original task

01 Deployed Demo

- Provide a deployed demo using Vercel, Netlify, or an equivalent platform.
- The demo should show: employee list, search/filter, add employee, edit employee, deactivate employee, and loading/empty/error states.

02 GitHub Repository

- Provide a GitHub repository using TypeScript with a clean and understandable structure.
- You may use any libraries or tools you prefer. There are no restrictions on UI libraries, state management, form libraries, data-fetching libraries, or styling solutions.
- You may use mock data or a mock API. No backend implementation is required.

03 The Senior Touch

- Include a brief README.md explaining how you would handle the employee API being unavailable.
- Explain how you would approach the page if the company had 100,000+ employees.

04 AI Usage Disclosure

- You are encouraged to use GPT, Claude, Copilot, Cursor, or other AI tools.
- Please include a PROMPTS.md file containing the main prompts you used during the implementation.

Product functionality you must build

Build an Employee Management Page in React and TypeScript. The original task does not require a backend. Mock data or a mock API is allowed.

01 Employee List

- Display: Name, Employee ID, Job Title, Department, Status, Joining Date, and Actions.
- Search by employee name.
- Filter by department.
- Filter by employment status.
- Pagination.

02 Employee Form

- Implement a form for adding and editing employees.
- Fields: First Name, Last Name, Email, Job Title, Department, Employment Status, and Joining Date.
- Add appropriate validation for required fields and email format.
- The same form should be reusable for both Create and Edit.

03 State Management & Data Flow

- Keep employee data and UI state maintainable and avoid unnecessary prop drilling.
- Handle loading state while fetching employees.
- Handle an empty state when no employees match the filters.
- Handle an error state when fetching employees fails.
- Handle loading state during create/edit operations.

04 Employee Actions

- View employee details.
- Edit an employee.
- Deactivate an employee.
- Deactivation should require a confirmation before the action is performed.

Remaining requirements and required files

05 Responsive UI

- The page should work on both desktop and mobile.
- The choice of responsive layout and styling approach is completely up to you.

06 Error Handling

- Demonstrate how the UI behaves when an API operation fails.
- Provide a clear recovery mechanism such as a Retry button or allowing the user to try the action again.

07 Accessibility

- Form fields have accessible labels.
- Buttons have meaningful accessible names.
- Main interactions are keyboard accessible.
- Validation errors are clearly communicated.

08 Testing

- Include at least one meaningful automated test demonstrating your testing approach.

STORYBOOK · NICE TO HAVE · NOT REQUIRED

Adding Storybook is not required, but it is a nice addition. If you choose to include it, showcase key reusable components and their main states, such as loading, empty, error, and populated states.

SENIOR TOUCH · README.md

- Include a brief README.md explaining how you would handle the employee API being unavailable.
- Explain how you would approach the page if the company had 100,000+ employees.

AI USAGE · PROMPTS.md

- You are encouraged to use GPT, Claude, Copilot, Cursor, or other AI tools.
- Please include a PROMPTS.md file containing the main prompts you used during the implementation.

LIVE DEMO MUST SHOW

Employee list | Search / filter | Add employee | Edit employee | Deactivate | Loading / empty / error

SUBMITTED CLIENT PACK

Live demo https://employee-management-eta-one.vercel.app

GitHub https://github.com/AbdullahYaseen01/employee-management

Also included: README.md Senior Touch · PROMPTS.md AI disclosure